

1

El Pattern Repositorio Como SSOT

El repositorio se considera la única fuente de verdad porque toda la aplicación incluyendo la UI y ViewModel, consulta los datos exclusivamente a través de el `DataService` de forma directa a `Room`. También es fachado porque expone una interfaz simple, que oculta toda la complejidad interna de coordinar 2 fuentes distintas.

Corrutinas y Structured Concurrency

El `viewModelScope` es un `CoroutineScope` vinculado al ciclo de vida del `viewModel`. Es fundamental porque cancela automáticamente todas las corrutinas que se están ejecutando dentro de el cuando el `viewModel` se destruye como cuando cerramos la pantalla o limpiamos memoria.

Flujos reactivos (cold vs. Hot)

Un flujo convencional es frío porque no emite datos hasta que haya un recolector activo repitiendo la ejecución lógica por cada suscripción. En cambio un `StateFlow` es caliente porque produce y mantiene un estado en memoria de manera independiente a si hay o no recolectores activos reteniendo siempre el último valor.

Inyección de dependencias

Se crea una instancia única de las dependencias globales (como la base de datos o el repositorio) dentro de una clase personalizada que extiende de aplicación.

El uso de `lazy` permite la inicialización perezosa. El objeto no se crea cuando la aplicación arranca, sino únicamente la primera vez que es utilizada. Esto optimiza el tiempo de inicio de la aplicación y ahorra memoria innecesaria si es que no hay uso.

2. Sección de Completar

1. El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo **Flow**.
2. El operador ^{Dispatchers.Default} permite ejecutar tareas pesadas fuera del MainThread, específicamente optimizado para tareas de CPU como el paseos masivo de datos.
3. Para que mi servicio en primer plano de ubicación sea válido en Android 10, debe declararse en el manifiesto con el tipo **Location**.
4. El delegado **by lazy** de la clase **Application** es el que garantiza que los componentes compartan la misma instancia del repositorio mediante un cast por ejemplo (**Application as MyApplication**).
5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama **suspend (Cancellable Coroutine)**.
6. El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente **AndroidManifest.xml**.
7. En la arquitectura AOSP la capa que actúa como puente estándar entre el framework y los controladores se denomina **HAL**.
8. El operador de Flow que permite combinar múltiples fuentes (GPS y sensores) para emitir un estado unificado es **Combine**.

3. Desarrollo de código y Lógica Personalizada.

Repositorio y DI

```
class UsuarioRepository (private val usuarioDAO: UsuarioDAO) {  
    suspend fun insertar Usuario (usuario: Usuario) {  
        usuarioDAO.insertar (usuario)  
    }  
    fun obtener Usuarios(): Flow<List<Usuario>> {  
        return usuarioDAO.obtener Todos ()  
    }  
}  
class MyApplication : Application {  
    private val database by lazy {  
        Room.databaseBuilder (this, AppDatabase::class, java,  
            "app.db").build ()  
    }  
    val usuarioRepository: UsuarioRepository by lazy {  
        UsuarioRepository (database.usuarioDAO ())  
    }  
}
```

Lógica dinámica de identidad

```
data class Usuario {  
    val nombre : String  
    val apellido : String  
    val edad : Int
```

```
}  
fun calcularPuntajeIdentidad (usuario : Usuario): Int {  
    return ( usuario.nombre.length : 0) + (usuario.apellidos.length : 0)
```

```
fun main() {
```

```
    val usuario = Usuario (nombre = "Victor", apellidos = "Faifan Aguilar",  
        edad = 23)
```

```
    val puntajeIdentidad = calcularPuntajeIdentidad (usuario)
```

```
    println ("usuario :#{usuario.nombre}/#{usuario.apellidos},
```

```
    tu puntaje de identidad es :#{puntajeIdentidad}")
```

```
}
```

5.

1. APLICACIONES

2. Marco de Trabajo de APLICACIONES

3a. Bibliotecas Nativas

3b. Android Runtime (ART)

4. CAPA DE ABSTRACCIÓN DE HARDWARE (HAL)

5. KERNEL DE LINUX

APLICACIONES : Capa visible para el usuario, compuesta por las apps del sistema y terceros instalados desde tienda.

Marco de trabajo de APLICACIONES :
• Expone en Kotlin y Java las APIs que los desarrolladores usan para construir.

Bibliotecas Nativas y Android Runtime